HyperAgent Complete Refactor: Full Setup Guide

Architecture, Diagrams, and Implementation Structure

Document Version: 1.0
Date: January 18, 2026
Target Completion: February 15, 2026
Total Timeline: 4 weeks (160 hours)

Table of Contents

1. Architecture Overview
2. System Diagrams & Flows
3. Project Structure
4. Setup & Installation
5. Week-by-Week Execution Plan
6. Type System & State Management
7. Node Implementation Templates
8. Testing Strategy
9. Deployment Architecture
10. Cost & Resource Tracking

Architecture Overview

High-Level System Architecture

```
┌──────────────
```

```
│ LangGraph State Machine │
│ (Specification-Locked Flow) │
└──────────────┬──────────────────────────────────────┘
               │
┌──────────────┴──────────────────────────────────────────────────────┐
│                                                                     │
▼                                                                     ▼
┌──────────────────┐                                  ┌──────────────────────┐
│ PolicyNode │                                  │ HyperAgentState │
│ (Parse) │──────────┐                       │ (7 Fields) │
└──────────────────┘                          │ │ │
▼                                             │ 1. userInput │
┌──────────────────────┐                      │ 2. policies │
│ GenerateNode │                              │ 3. generatedCode │
│ (Code Gen) │──────────────┬──────────────▶│ 4. auditResults │
│ Claude Opus │                              │ 5. validationSt. │
│ (+ fallback) │                              │ 6. deploymentH. │
└──────────────────────┘                      │ 7. monitoringM. │
│                                             └──────────────────────┘
▼                                             │
┌──────────────────────┐                      │
│ AuditNode │                                 │
│ (Security) │──────────┤
│ Slither + │                                 │
│ Claude │                                    │
└──────────────────────┘                      │
│                                             │
▼                                             │
┌──────────────────────┐                      │
│ ValidateNode │                              │
│ (Verify State)│──────────┘
│ Check 7 │
│ fields exist │
└──────────────────────┘
│
┌──────────────────────┼──────────────────────┐
│                      │                      │
▼                      ▼                      ▼
┌──────────────┐  ┌──────────────────┐  ┌──────────────────┐
│ Deploy │  │ Monitor │  │ EigenDA │
│ Node │  │ Node │  │ Node │
│ (Chain) │  │ (Tracking) │  │ (Storage) │
└──────┬───────┘  └──────────┬───────────┘  └──────────┬───────────┘
│                      │                      │
▼                      ▼                      ▼
┌──────────────┐  ┌──────────────────┐  ┌──────────────────┐
│Mantle │  │Prometheus │  │Pinata IPFS │
│Ethereum │  │Grafana │  │Blockchain │
│Polygon │  │Dashboard │  │Registry │
└──────────────┘  └──────────────────┘  └──────────────────┘
│
```

▼

```
┌──────────────────────────┐
│ CONTRACT DEPLOYED │
│ + Audit Trail │
│ + Monitoring Active │
│ + Immutable Storage │
└──────────────────────────┘
```

Component Interaction Matrix

```
┌──────────┬──────────┬──────────┬──────────┬──────────┬──────────┬──────────
│ Policy   │ Generate │ Audit    │ Validate │ Deploy   │ Monitor  │ EigenDA
│ Node     │ Node     │ Node     │ Node     │ Node     │ Node     │ Node     │
```

```
├──────────────────┼──────────┼──────────┼──────────┼──────────┼──────────┼
──────────┼──────────┼──────────┤
│Input Type │ string │ policies │ code │ full │ code │ address │ audit │
│ │ │ (intent) │ │ state │ + │ + state │ trail │
│ │ │ │ │ │ chain │ │ │
├──────────────────┼──────────┼──────────┼──────────┼──────────┼──────────┼
──────────┼──────────┼──────────┤
│Output Type │ policies │ code │ audit │ valid │ deploy │ metrics │ proof │
│ │ │ + hash │ report │ (bool) │ tx │ (json) │ (hash) │
├──────────────────┼──────────┼──────────┼──────────┼──────────┼──────────┼
──────────┼──────────┼──────────┤
│LLM Used │ None │ Opus → │ Sonnet │ None │ None │ None │ None │
│ │ (regex) │ Flash → │ (opt) │ │ │ │ │
│ │ │ Turbo │ │ │ │ │ │
├──────────────────┼──────────┼──────────┼──────────┼──────────┼──────────┼
──────────┼──────────┼──────────┤
│Memory Access │ None │ Chroma │ None │ State │ None │ Prom │ IPFS + │
│ │ │ (vectors)│ │ dict │ │ Grafana │ Blockchain│
├──────────────────┼──────────┼──────────┼──────────┼──────────┼──────────┼
──────────┼──────────┼──────────┤
│Error Handling │ Regex │ LLM │ Slither │ Type │ Chain │ Network │ Network │
│ │ fails │ fallback │ fails │ mismatch │ fails │ errors │ errors │
├──────────────────┼──────────┼──────────┼──────────┼──────────┼──────────┼
──────────┼──────────┼──────────┤
│Async? │ No │ Yes │ Yes │ No │ Yes │ Yes │ Yes │
│ │ (fast) │ (network)│ (network)│ (local) │(network)│(network)│(network) │
└──────────────────┴──────────┴──────────┴──────────┴──────────┴──────────┴
──────────┴──────────┴──────────┘
```

System Diagrams & Flows

1. State Machine Flow Diagram (LangGraph)

```
START
│
├─ Receive userInput string
│
▼
┌─────────────────────────────────────────────────────────────┐
│ HyperAgentState Created │
│ │
│ { │
│ userInput: "Create ERC-20 token", │
│ policies: [], │
│ generatedCode: "", │
│ auditResults: {}, │
│ validationStatus: false, │
│ deploymentHash: "", │
│ monitoringMetrics: {} │
│ } │
└─────────────────────────────────────────────────────────────┘
│
▼
POLICY NODE
├─ Input: userInput (string)
├─ Process: Extract requirements (regex)
├─ Output: policies array
└─ State Update: policies = [...]
│
▼
GENERATE NODE
├─ Input: policies array
├─ Query: Chroma for similar contracts
├─ LLM Call: Claude Opus (primary)
│ ├─ Success → use response
│ ├─ Fallback → Gemini 2.0 Flash
│ ├─ Fallback → GPT-4 Turbo
│ └─ Fallback → Claude Sonnet
├─ Output: generated Solidity code
└─ State Update: generatedCode = code_string
│
▼
AUDIT NODE
├─ Input: generatedCode (string)
├─ Process: Run Slither static analysis
│ ├─ HIGH severity → FAIL (loop back to Generate)
│ ├─ MEDIUM severity → flag for Claude review
│ └─ LOW severity → accept
├─ Output: audit report (JSON)
```

```
└─ State Update: auditResults = report_json
│
▼ (if audit passes)
VALIDATE NODE
├─ Input: full HyperAgentState
├─ Verify: All 7 fields present
├─ Verify: All types correct (TypeScript strict)
├─ Verify: No extra fields (spec-locked)
├─ Output: validationStatus (boolean)
└─ State Update: validationStatus = true/false
│
├─ IF FALSE → ERROR (something corrupted state)
│
▼ (if validation passes)
DEPLOY NODE
├─ Input: generatedCode + user wallet + chain choice
├─ Process:
│  ├─ Compile with solc
│  ├─ Create ethers.js contract object
│  ├─ Sign transaction with user key
│  ├─ Submit to Mantle (primary)
│  └─ Wait for confirmation (12 blocks)
├─ Output: deploymentHash (transaction hash)
└─ State Update: deploymentHash = "0x..."
│
▼
MONITOR NODE
├─ Input: deploymentHash + contract address
├─ Start background tracking:
│  ├─ Poll blockchain state every 30s
│  ├─ Emit Prometheus metrics
│  ├─ Track gas usage
│  └─ Log any errors/events
├─ Output: monitoring report (JSON)
└─ State Update: monitoringMetrics = {...}
│
▼
EIGENDA NODE
├─ Input: entire HyperAgentState + audit trail
├─ Process:
│  ├─ Serialize HyperAgentState to JSON
│  ├─ Upload to Pinata IPFS
│  ├─ Get IPFS hash
│  ├─ Store hash on Mantle registry contract
│  └─ Create merkle proof
├─ Output: proof object (IPFS hash + blockchain proof)
└─ State Update: (immutable, only reads state)
│
▼
RETURN OUTPUT
├─ Contract address
```

```
├─ Deployment transaction hash
├─ Full audit trail (IPFS)
├─ Monitoring dashboard link
└─ Immutable proof of execution
│
▼
END ✓
```

2. Data Flow Diagram (End-to-End)

```
USER INTERFACE
│
│ POST /api/contracts/generate
├─ Body: {
│ userInput: "Create ERC-20 with minting limits",
│ walletAddress: "0x...",
│ chain: "mantle"
│ }
│
▼
API GATEWAY (Express/Fastify)
├─ Validate input schema
├─ Check rate limits
├─ Authenticate user
│
▼
REQUEST HANDLER
├─ Initialize HyperAgentState
├─ Create LangGraph workflow runner
│
▼
LANGGRAPH STATE MACHINE
├─ Node 1: PolicyNode
│ ├─ Read: state.userInput
│ ├─ Process: Extract requirements
│ └─ Write: state.policies
│
├─ Node 2: GenerateNode
│ ├─ Read: state.policies
│ ├─ Query: Chroma vector DB (similar contracts)
│ ├─ Call: LLM with fallback chain
│ │ ├─ Claude Opus (cost: $0.015)
│ │ ├─ Gemini Flash (cost: free)
│ │ ├─ GPT-4 Turbo (cost: $0.010)
│ │ └─ Claude Sonnet (cost: $0.003)
│ ├─ Cache: Store in Chroma for future reference
│ └─ Write: state.generatedCode
│
├─ Node 3: AuditNode
│ ├─ Read: state.generatedCode
│ ├─ Run: Slither analysis (static)
```

```
│  ├─ Check: Block dangerous patterns
│  │  ├─ reentrancy
│  │  ├─ overflow/underflow
│  │  ├─ delegatecall abuse
│  │  └─ etc.
│  ├─ Optional: Claude review (if medium severity)
│  └─ Write: state.auditResults
│
├─ Node 4: ValidateNode
│  ├─ Read: entire state
│  ├─ Verify: 7 fields present
│  ├─ Verify: Types match spec
│  ├─ Verify: No corruption
│  └─ Write: state.validationStatus
│
├─ Node 5: DeployNode
│  ├─ Read: state.generatedCode, walletAddress, chain
│  ├─ Compile: Solidity → bytecode
│  ├─ Sign: ethers.js transaction
│  ├─ Submit: To Mantle RPC endpoint
│  ├─ Wait: 12 block confirmations
│  └─ Write: state.deploymentHash
│
├─ Node 6: MonitorNode
│  ├─ Read: state.deploymentHash, contract address
│  ├─ Start: Background job (async)
│  ├─ Poll: Blockchain state every 30s
│  ├─ Track: Gas usage, events, errors
│  ├─ Emit: Prometheus metrics
│  └─ Write: state.monitoringMetrics
│
└─ Node 7: EigenDANode
   ├─ Read: entire state (immutable)
   ├─ Serialize: HyperAgentState → JSON
   ├─ Upload: To Pinata IPFS
   ├─ Store: Hash on Mantle registry
   └─ Return: IPFS proof hash

RESPONSE BACK TO USER
├─ Contract address: 0x...
├─ Deployment tx: 0x...
├─ Audit trail IPFS: QmXxx...
├─ Monitoring dashboard: https://...
└─ Status: ✓ SUCCESS
```

3. LLM Fallback Chain Decision Tree

GenerateNode receives: state.policies

```
┌─────────────────────────┐
│ Try Claude Opus         │
│ (Best quality)          │
│ Cost: $0.015            │
└─────────────┬───────────┘
              │
        ┌─────┴─────┐
        │           │
    SUCCESS       FAILURE
        │           │
        ▼           ▼
✓ Return Code ┌─────────────────────┐
│ Try Gemini 2.0 Flash│
│ (Fast, Free) │
│ Cost: $0 │
└─────────────┬─────────────────┘
│
┌─────┴─────┐
│ │
SUCCESS FAILURE
│ │
▼ ▼
✓ Return Code ┌─────────────────────┐
│ Try GPT-4 Turbo │
│ (Alternative) │
│ Cost: $0.010 │
└─────────────┬─────────────────┘
│
┌─────┴─────┐
│ │
SUCCESS FAILURE
│ │
▼ ▼
✓ Return Code ┌─────────────────────┐
│ Claude Sonnet │
│ (Budget option) │
│ Cost: $0.003 │
└─────────────┬─────────────────┘
│
┌─────┴─────┐
│ │
```

```
SUCCESS FAILURE
│ │
▼ ▼
✓ Return Code ✗ RETURN ERROR
│
Send to monitoring
Log for review

COST OPTIMIZATION:
├─ Average per call: $0.004 (due to fallbacks working)
├─ Success rate: 99%+ (fallback chain catches failures)
└─ User cost: $0.020 per contract (with margin)
```

4. Type Safety & Verification Flow

```
REQUEST ARRIVES
│
├─ Schema Validation (Express middleware)
│  ├─ Body shape correct? (Zod schema)
│  ├─ All required fields? (userInput, walletAddress, chain)
│  └─ Types match? (string, string, enum)
│  └─ FAIL → 400 Bad Request
│
▼
INITIALIZE STATE (TypeScript)
│
├─ Create HyperAgentState object:
│ {
│ userInput: string, ← Type checked by TypeScript
│ policies: string[], ← Must be array of strings
│ generatedCode: string, ← Must be string
│ auditResults: AuditReport, ← Custom type
│ validationStatus: boolean, ← Must be boolean
│ deploymentHash: string, ← Must be string
│ monitoringMetrics: MetricsObject ← Custom type
│ }
│
├─ TypeScript Compiler (tsc)
│  ├─ Check: All 7 fields present? (compile error if not)
│  ├─ Check: No extra fields? (compile error if added)
│  ├─ Check: Types match? (compile error if wrong)
│  └─ PASS → Continue
│
▼
NODE EXECUTION (Each Node)
│
├─ Input Validation
│  ├─ Node receives state (type-checked ✓)
│  ├─ Extracts required fields
│  ├─ Type guards in TypeScript catch errors early
│  └─ PASS → Process
```

```
│
├─ Processing
│  ├─ Perform node-specific logic
│  ├─ Update specific state fields
│  └─ Other fields remain unchanged (immutable principles)
│
└─ Output Validation
├─ Before writing to state, validate new value:
│  ├─ PolicyNode output: policies must be string[] of valid JSON
│  ├─ GenerateNode output: generatedCode must be valid Solidity
│  ├─ AuditNode output: auditResults must match AuditReport schema
│  └─ etc. for each node
├─ TypeScript checks compile-time
└─ PASS → Update state

SPECIFICATION DICTIONARY VERIFICATION
│
├─ After full state machine completes:
│  ├─ Read: Complete HyperAgentState
│  ├─ Verify: All 7 fields present (no hallucinations)
│  ├─ Verify: All types correct (no corruption)
│  ├─ Verify: No extra fields (spec-locked)
│  ├─ Verify: Values make sense (cross-field validation)
│  └─ Report: Pass/Fail with detailed error if fail
│
├─ Cross-Field Validation Rules:
│  ├─ IF deploymentHash is not empty → contract exists
│  ├─ IF validationStatus is true → all fields populated
│  ├─ IF auditResults.severity === HIGH → generatedCode has issue
│  └─ etc.
│
▼
RESPONSE TO USER
│
├─ IF all verifications pass:
│  └─ Return: Contract address + proof ✓
│
└─ IF any verification fails:
└─ Return: 500 Error with detailed diagnostic
```

Project Structure

Complete Directory Tree

```
hyperagent/
│
├── src/ # Source code (TypeScript)
│  │
│  ├── types/ # Type definitions
│  │  ├── state.ts # HyperAgentState (7 fields)
```

```
│ │ ├── nodes.ts # Node input/output types
│ │ ├── blockchain.ts # Chain-specific types
│ │ ├── audit.ts # Audit report schema
│ │ ├── monitoring.ts # Metrics types
│ │ └── index.ts # Export all types
│ │
│ ├── nodes/ # 7 Node Implementations
│ │ ├── policy.ts # PolicyNode
│ │ │ ├─ parseUserInput()
│ │ │ ├─ extractRequirements()
│ │ │ └─ generatePolicies()
│ │ │
│ │ ├── generate.ts # GenerateNode
│ │ │ ├─ queryChroma()
│ │ │ ├─ generateCode()
│ │ │ ├─ llmRouter()
│ │ │ └─ cacheInChroma()
│ │ │
│ │ ├── audit.ts # AuditNode
│ │ │ ├─ runSlither()
│ │ │ ├─ checkBlockedPatterns()
│ │ │ └─ claudeReview()
│ │ │
│ │ ├── validate.ts # ValidateNode
│ │ │ ├─ verifyStateFields()
│ │ │ ├─ typeCheck()
│ │ │ └─ crossFieldValidation()
│ │ │
│ │ ├── deploy.ts # DeployNode
│ │ │ ├─ compileContract()
│ │ │ ├─ signTransaction()
│ │ │ ├─ submitToChain()
│ │ │ └─ waitForConfirmation()
│ │ │
│ │ ├── monitor.ts # MonitorNode
│ │ │ ├─ startTracking()
│ │ │ ├─ pollBlockchain()
│ │ │ ├─ emitMetrics()
│ │ │ └─ handleErrors()
│ │ │
│ │ ├── eigenda.ts # EigenDANode
│ │ │ ├─ serializeState()
│ │ │ ├─ uploadToIPFS()
│ │ │ ├─ storeOnBlockchain()
│ │ │ └─ generateProof()
│ │ │
│ │ └── index.ts # Export all nodes
│ │
│ ├── protocols/ # Core Protocols
│ │ ├── verification.ts # Type-safe verification
│ │ │ ├─ verifyHyperAgentState()
```

```
│ │ │ ├─ checkDictionary()
│ │ │ └─ validateSpecCompliance()
│ │ │
│ │ ├── state-machine.ts # LangGraph workflow
│ │ │ ├─ createWorkflow()
│ │ │ ├─ defineStateGraph()
│ │ │ ├─ addNodes()
│ │ │ └─ defineTransitions()
│ │ │
│ │ ├── llm-routing.ts # LLM fallback chain
│ │ │ ├─ selectLLM()
│ │ │ ├─ handleFallback()
│ │ │ ├─ trackCost()
│ │ │ └─ retryLogic()
│ │ │
│ │ └── index.ts # Export all protocols
│ │
│ ├── services/ # External Service Integrations
│ │ │
│ │ ├── llm/ # LLM Service
│ │ │ ├── claude-opus.ts # Anthropic Opus (primary)
│ │ │ ├── gemini-flash.ts # Google Gemini (fallback 1)
│ │ │ ├── gpt4-turbo.ts # OpenAI GPT-4 (fallback 2)
│ │ │ ├── claude-sonnet.ts # Anthropic Sonnet (fallback 3)
│ │ │ └── router.ts # Route to appropriate LLM
│ │ │
│ │ ├── storage/ # Storage Service
│ │ │ ├── chroma.ts # Vector database (Chroma)
│ │ │ │ ├─ initialize()
│ │ │ │ ├─ storeEmbedding()
│ │ │ │ ├─ queryByVector()
│ │ │ │ └─ updateCollection()
│ │ │ │
│ │ │ ├── pinata.ts # IPFS storage (Pinata)
│ │ │ │ ├─ uploadJSON()
│ │ │ │ ├─ retrieveFromIPFS()
│ │ │ │ └─ pinFile()
│ │ │ │
│ │ │ └── blockchain.ts # Blockchain storage
│ │ │ ├─ storeHash()
│ │ │ ├─ retrieveHash()
│ │ │ └─ verifyProof()
│ │ │
│ │ ├── blockchain/ # Blockchain Service
│ │ │ ├── provider.ts # RPC providers (multi-chain)
│ │ │ ├── mantle.ts # Mantle L2 (primary)
│ │ │ ├── ethereum.ts # Ethereum L1 (secure)
│ │ │ ├── polygon.ts # Polygon (cost-effective)
│ │ │ └── signer.ts # Transaction signing
│ │ │
│ │ └── monitoring/ # Monitoring Service
```

```
│ │ ├── prometheus.ts # Metrics collection
│ │ │ ├─ registerMetric()
│ │ │ ├─ recordMetric()
│ │ │ └─ getMetrics()
│ │ │
│ │ └── grafana.ts # Dashboard integration
│ │ ├─ createDashboard()
│ │ ├─ addChart()
│ │ └─ setAlerts()
│ │
│ ├── memory/ # Memory Management
│ │ ├── vector-store.ts # Chroma integration
│ │ │ ├─ embedContract()
│ │ │ ├─ findSimilar()
│ │ │ └─ cacheResult()
│ │ │
│ │ ├── contract-cache.ts # Fast lookups
│ │ │ ├─ get()
│ │ │ ├─ set()
│ │ │ └─ invalidate()
│ │ │
│ │ └── pattern-library.ts # Reusable patterns
│ │ ├─ registerPattern()
│ │ ├─ getPattern()
│ │ └─ matchPatterns()
│ │
│ ├── api/ # REST API
│ │ ├── main.ts # Server initialization
│ │ │ ├─ createApp()
│ │ │ ├─ setupMiddleware()
│ │ │ └─ startServer()
│ │ │
│ │ ├── routes.ts # API endpoints
│ │ │ ├─ POST /api/contracts/generate
│ │ │ ├─ GET /api/contracts/:id
│ │ │ ├─ GET /api/contracts/:id/audit
│ │ │ ├─ GET /api/monitoring/:id
│ │ │ └─ GET /api/health
│ │ │
│ │ └── middleware.ts # Express middleware
│ │ ├─ validateSchema()
│ │ ├─ rateLimit()
│ │ ├─ authenticate()
│ │ ├─ errorHandler()
│ │ └─ requestLogger()
│ │
│ ├── utils/ # Utilities
│ │ ├── cost-calculator.ts # Track per-component costs
│ │ │ ├─ calculateLLMCost()
│ │ │ ├─ calculateGasCost()
│ │ │ └─ getTotalCost()
```

```
│ │ │
│ │ ├── audit-logger.ts # Immutable audit trail
│ │ │ ├─ logAction()
│ │ │ ├─ getAuditTrail()
│ │ │ └─ exportTrail()
│ │ │
│ │ ├── error-handler.ts # Consistent error handling
│ │ │ ├─ createError()
│ │ │ ├─ formatErrorResponse()
│ │ │ └─ logError()
│ │ │
│ │ └── helpers.ts # Common utilities
│ │ ├─ sleep()
│ │ ├─ retry()
│ │ └─ parseJSON()
│ │
│ └── index.ts # Entry point
│ ├─ Initialize app
│ ├─ Load configuration
│ └─ Start server
│
├── tests/ # Test Suite
│ │
│ ├── unit/ # Unit tests (90%+ coverage)
│ │ ├── policy.test.ts
│ │ ├── generate.test.ts
│ │ ├── audit.test.ts
│ │ ├── validate.test.ts
│ │ ├── deploy.test.ts
│ │ ├── monitor.test.ts
│ │ ├── eigenda.test.ts
│ │ ├── verification.test.ts
│ │ ├── state-machine.test.ts
│ │ └── llm-routing.test.ts
│ │
│ ├── integration/ # Integration tests
│ │ ├── e2e-flow.test.ts # Full workflow
│ │ ├── node-interactions.test.ts # Node communication
│ │ ├── state-transitions.test.ts # State machine flow
│ │ └── error-handling.test.ts # Error scenarios
│ │
│ ├── fixtures/ # Test data
│ │ ├── contracts.json # Sample contracts
│ │ ├── states.json # Sample states
│ │ └── audit-reports.json # Sample audit results
│ │
│ └── setup.ts # Jest configuration
│
├── docs/ # Documentation (7 files)
│ ├── hyperagent_dna_blueprint.md # Node specifications
│ ├── hyperagent_playbook.md # Day-by-day timeline
```

```
│  ├── hyperagent_system_prompts.md # LLM safety protocols
│  ├── hyperagent_tech_stack_architecture.md
│  ├── tech_stack_blueprint_integration.md
│  ├── documentation_index.md # Navigation guide
│  ├── blueprint_executive_summary.md # Quick reference
│  └── complete_system_guide.md # Full guide
│
├── public/ # Static assets
│  ├── index.html
│  └── styles.css
│
├── config/ # Configuration files
│  ├── prometheus.yml # Prometheus config
│  ├── grafana-dashboard.json # Grafana dashboard
│  └── .env.example # Environment template
│
├── scripts/ # Build & deployment scripts
│  ├── build.sh # Build TypeScript
│  ├── test.sh # Run tests
│  ├── deploy.sh # Deploy to Render
│  └── verify.sh # Run verification
│
├── docker/ # Docker configuration
│  ├── Dockerfile # Node.js image
│  ├── docker-compose.yml # Services definition
│  └── .dockerignore
│
├── .github/ # GitHub Actions CI/CD
│  └── workflows/
│  ├── test.yml # Run tests on push
│  ├── deploy.yml # Deploy on release
│  └── verify.yml # Verify spec compliance
│
├── package.json # npm dependencies
├── tsconfig.json # TypeScript configuration
├── jest.config.js # Jest testing framework
├── .eslintrc.json # ESLint configuration
├── .prettierrc.json # Code formatting
├── README.md # Quick start guide
├── LICENSE # MIT License
├── .gitignore # Git ignore rules
└── CONTRIBUTING.md # Contribution guide
```

Setup & Installation

Prerequisites

Node.js version

```
node --version # Should be >= 20.0.0
```

npm version

```
npm --version # Should be >= 10.0.0
```

Git

```
git --version
```

Step 1: Initialize Repository

Create project directory

```
mkdir hyperagent-refactor
cd hyperagent-refactor
```

Initialize git

```
git init
git config user.name "Your Name"
git config user.email "your.email@example.com"
```

Create .gitignore

```
cat > .gitignore << 'EOF'
node_modules/
dist/
*.log
.env
.env.local
.DS_Store
coverage/
.vscode/
.idea/
*.swp
EOF
```

Create initial commit

```
git add .gitignore
git commit -m "Initial commit: project setup"
```

Step 2: TypeScript & Build Setup

Initialize npm project

```
npm init -y
```

Install TypeScript & build tools

```
npm install --save-dev
typescript
@types/node
ts-node
tslib
```

Create tsconfig.json

```
cat > tsconfig.json << 'EOF'
{
"compilerOptions": {
"target": "ES2020",
"module": "commonjs",
"lib": ["ES2020"],
"outDir": "./dist",
"rootDir": "./src",
"strict": true,
"esModuleInterop": true,
"skipLibCheck": true,
"forceConsistentCasingInFileNames": true,
"resolveJsonModule": true,
"declaration": true,
"declarationMap": true,
"sourceMap": true,
"noImplicitAny": true,
"strictNullChecks": true,
"strictFunctionTypes": true,
"noUnusedLocals": true,
"noUnusedParameters": true,
"noImplicitReturns": true
},
"include": ["src"],
"exclude": ["node_modules", "dist", "tests"]
}
EOF
```

Verify TypeScript installation

npx tsc --version

Step 3: Testing Framework Setup

Install Jest

npm install --save-dev
jest
@types/jest
ts-jest

Create jest.config.js

cat > jest.config.js << 'EOF'
module.exports = {
preset: 'ts-jest',
testEnvironment: 'node',
roots: ['<rootDir>/tests'],
testMatch: ['/**.test.ts'],
collectCoverageFrom: [
'src//*.ts',*
*'!src/***/.d.ts',
'!src/index.ts'
],
coverageThreshold: {
global: {
branches: 80,
functions: 90,
lines: 90,
statements: 90
}
}
};
EOF

Verify Jest installation

npx jest --version

Step 4: LangGraph & Dependencies

Install LangGraph

npm install
@langchain/langgraph
@langchain/core
langchain

Install LLM providers

npm install
@anthropic-ai/sdk
@google/generative-ai
openai

Install blockchain libraries

npm install
ethers@v6
@ethers-ext/ethers

Install storage & monitoring

npm install
chroma-js
pinata
prom-client
dotenv
axios

Install Express & validation

npm install express zod
npm install --save-dev @types/express

Verify all installations

npm list

Step 5: Create Environment File

Copy environment template

cp .env.example .env

Edit .env with your keys

cat > .env << 'EOF'

API Keys

ANTHROPIC_API_KEY=sk-ant-...
GOOGLE_API_KEY=...
OPENAI_API_KEY=sk-...

Blockchain

MANTLE_RPC_URL=https://rpc.mantle.xyz
ETHEREUM_RPC_URL=https://eth-mainnet.g.alchemy.com/v2/...
POLYGON_RPC_URL=https://polygon-rpc.com

Storage

PINATA_API_KEY=...
PINATA_API_SECRET=...
CHROMA_HOST=localhost
CHROMA_PORT=8000

Monitoring

PROMETHEUS_PORT=9090
GRAFANA_PORT=3000

Application

NODE_ENV=development
PORT=3000
LOG_LEVEL=info
EOF

Add .env to .gitignore (don't commit secrets!)

```
echo ".env" >> .gitignore
```

Step 6: Create Source Directory Structure

Create directory tree

```
mkdir -p
src/{types,nodes,protocols,services/{llm,storage,blockchain,monitoring},memory,api,utils}
mkdir -p tests/{unit,integration,fixtures}
mkdir -p config
mkdir -p docker
mkdir -p scripts
mkdir -p docs
mkdir -p public
```

Create index files

```
touch src/types/index.ts
touch src/nodes/index.ts
touch src/protocols/index.ts
touch src/services/index.ts
touch src/memory/index.ts
touch src/api/index.ts
touch src/utils/index.ts
touch src/index.ts
```

Verify structure

```
tree -L 3 src/
```

Step 7: Add npm Scripts

Update package.json scripts

```
cat >> package.json << 'EOF'
"scripts": {
"build": "tsc",
"start": "node dist/index.js",
"dev": "ts-node src/index.ts",
"test": "jest",
"test:watch": "jest --watch",
"test:coverage": "jest --coverage",
"lint": "eslint src tests",
```

```
"format": "prettier --write 'src//*.ts' 'tests//*.ts'",
"verify": "npm run lint && npm run test",
"docker:build": "docker build -t hyperagent:latest .",
"docker:up": "docker-compose up -d",
"docker:down": "docker-compose down"
}
EOF
```

Test the scripts

```
npm run build # Should create dist/ folder
npm test # Should run Jest
```

Week-by-Week Execution Plan

Week 1: Foundation (January 19-25)

Goal: Establish core infrastructure and PolicyNode

Day 1-2: Project Infrastructure

Tuesday, January 19

Morning: Setup complete

```
9:00 AM - 12:00 PM
├─ Clone existing repo (backup)
├─ Create new TypeScript project
├─ Install all dependencies
├─ Create directory structure
├─ Commit: "Project initialization"
└─ Verify: npm run build (no errors)
```

Afternoon: Type System Foundation

```
1:00 PM - 5:00 PM
├─ Create src/types/state.ts
│  └─ Define HyperAgentState type
├─ Create src/types/nodes.ts
│  └─ Define node input/output types
├─ Create src/types/blockchain.ts
│  └─ Define chain-specific types
├─ Add unit tests for types
├─ Commit: "Add type definitions"
└─ Verify: npm test (all pass)
```

Wednesday, January 20

Full day: Verification Protocol

```
├─ Morning: Build verification system
│  ├─ Create src/protocols/verification.ts
│  ├─ Implement verifyHyperAgentState()
│  ├─ Implement checkDictionary()
│  └─ Create comprehensive tests
│
├─ Afternoon: LangGraph Setup
│  ├─ Create src/protocols/state-machine.ts
│  ├─ Initialize LangGraph workflow
│  ├─ Define state graph structure
│  ├─ Test state transitions
│  └─ Commit: "Add verification protocol & state machine"
│
└─ EOD Checkpoint:
✓ State machine initialized
✓ All 7 node structures defined
✓ Verification protocol operational
✓ 95% test coverage
```

Day 3-5: PolicyNode Implementation

Thursday, January 21

Full day: PolicyNode

```
├─ Create src/nodes/policy.ts
├─ Implement parseUserInput() function
│  └─ Extract ERC standards, minting rules, etc.
├─ Implement extractRequirements() function
│  └─ Convert text to structured policies
├─ Implement generatePolicies() function
│  └─ Return array of policy objects
├─ Create tests/unit/policy.test.ts
│  ├─ Test regex parsing
│  ├─ Test requirement extraction
│  └─ Test policy generation
├─ Commit: "Add PolicyNode implementation"
└─ npm test -- policy.test.ts (all pass)
```

```
EXPECTED OUTPUT:
Input: "Create ERC-20 token with 10M max supply and 25% burn"
Output: {
policies: [
{ type: "erc-20", version: "latest" },
{ type: "max-supply", value: 10000000 },
{ type: "burn-rate", value: 0.25 }
]
}
```

Friday, January 22

Integrate PolicyNode with state machine

```
├─ Update src/protocols/state-machine.ts
│  └─ Add PolicyNode as first step
├─ Test PolicyNode → GenerateNode transition
│  └─ State passed correctly between nodes
├─ Integration test: Full input → policies output
│  └─ tests/integration/node-interactions.test.ts
├─ Commit: "Integrate PolicyNode into state machine"
└─ npm run test:coverage (90%+ overall)
```

CHECKPOINT:
✓ PolicyNode working
✓ State machine progressing
✓ Tests at 90%+ coverage
✓ Ready for GenerateNode

Weekend (Jan 25-26): Optional Buffer

- Code review
- Documentation updates
- Performance profiling
- Prepare for Week 2

Week 2: Core Nodes (January 26-February 1)

Goal: Implement GenerateNode, AuditNode, ValidateNode, DeployNode

Day 1-2: GenerateNode with LLM Router

Monday-Tuesday, January 26-27

GenerateNode: Most critical node

Step 1: LLM Router

```
├─ Create src/services/llm/router.ts
├─ Implement selectLLM() function
│  └─ Fallback chain logic
├─ Implement handleFallback() function
│  └─ Retry logic with exponential backoff
├─ Implement trackCost() function
│  └─ Calculate LLM cost per call
├─ Tests: Verify fallback chain triggers correctly
```

Step 2: Individual LLM Services

```
├─ Create src/services/llm/claude-opus.ts
│  └─ Call Anthropic API with specific model
├─ Create src/services/llm/gemini-flash.ts
│  └─ Call Google API with free-tier model
├─ Create src/services/llm/gpt4-turbo.ts
│  └─ Call OpenAI API
├─ Create src/services/llm/claude-sonnet.ts
│  └─ Call Anthropic cheaper model
```

Step 3: Chroma Integration

```
├─ Create src/memory/vector-store.ts
├─ Implement embedContract() function
│  └─ Convert contract code to vector embedding
├─ Implement findSimilar() function
│  └─ Query Chroma for similar contracts
├─ Implement cacheResult() function
│  └─ Store generated code in Chroma
```

Step 4: GenerateNode

```
├─ Create src/nodes/generate.ts
├─ Implement generateCode() function
│  ├─ Query Chroma for similar contracts
│  ├─ Call LLM with fallback chain
│  ├─ Validate output is valid Solidity
│  └─ Cache result in Chroma
├─ Create tests/unit/generate.test.ts
│  ├─ Test LLM selection
│  ├─ Test fallback chain
│  ├─ Test Chroma integration
│  └─ Test output validation
```

Step 5: Integration

```
├─ Update state-machine.ts
│  └─ Add GenerateNode after PolicyNode
├─ Test full flow: PolicyNode → GenerateNode
├─ Commit: "Add GenerateNode with LLM router"
└─ npm test (90%+ coverage maintained)

EXPECTED COST PER CALL:
├─ Claude Opus: $0.015 (best quality)
├─ If Opus fails → Gemini Flash: $0 (free)
├─ If Flash fails → GPT-4 Turbo: $0.010
```

```
├─ If Turbo fails → Claude Sonnet: $0.003
└─ Average: $0.004 per contract
```

Day 3: AuditNode

Wednesday, January 28

AuditNode: Security analysis

```
├─ Create src/nodes/audit.ts
├─ Implement runSlither() function
│  ├─ Call Slither static analysis tool
│  ├─ Parse results
│  ├─ Categorize severity (HIGH/MEDIUM/LOW)
│  └─ Block HIGH severity issues
├─ Implement checkBlockedPatterns() function
│  ├─ Reentrancy
│  ├─ Integer overflow/underflow
│  ├─ Delegatecall abuse
│  ├─ Unchecked send
│  └─ etc. (20+ patterns)
├─ Implement claudeReview() function (optional)
│  └─ For MEDIUM severity, get Claude opinion
├─ Create tests/unit/audit.test.ts
│  ├─ Test Slither integration
│  ├─ Test pattern detection
│  ├─ Test severity categorization
│  └─ Test error handling
```

Integration

```
├─ Update state-machine.ts
│  └─ Add AuditNode after GenerateNode
├─ Add loop-back: If HIGH severity → return to GenerateNode
├─ Test: GenerateNode → AuditNode → (if fail, back to Generate)
├─ Commit: "Add AuditNode with Slither"
└─ npm test (maintain 90%+ coverage)
```

```
EXPECTED BEHAVIOR:
Input: Generated Solidity code
├─ Run Slither analysis
├─ If HIGH severity found:
│  └─ Return FAIL + detailed error
│  → Trigger GenerateNode with feedback
├─ If MEDIUM severity found:
│  ├─ Get Claude opinion
│  ├─ If Claude agrees it's ok:
│  │  └─ Continue
│  └─ If Claude agrees it's bad:
│  └─ Return FAIL
```

└─ If LOW severity:
└─ Continue with warning in auditResults

Day 4: ValidateNode

Thursday, January 29

ValidateNode: State verification

├─ Create src/nodes/validate.ts
├─ Implement verifyStateFields() function
│ ├─ Check all 7 fields present
│ ├─ Check no extra fields (spec-locked)
│ └─ Return boolean: all present?
├─ Implement typeCheck() function
│ ├─ Use TypeScript type guards
│ ├─ Verify each field type matches
│ └─ Throw TypeError if mismatch
├─ Implement crossFieldValidation() function
│ ├─ IF deploymentHash exists → check validity
│ ├─ IF validationStatus true → check all required fields
│ ├─ IF auditResults exist → check format
│ └─ etc. (cross-field rules)
├─ Create tests/unit/validate.test.ts
│ ├─ Test field presence detection
│ ├─ Test type checking
│ ├─ Test cross-field validation
│ └─ Test error handling

Integration

├─ Update state-machine.ts
│ └─ Add ValidateNode after AuditNode
├─ Test: Full flow so far passes ValidateNode
├─ Commit: "Add ValidateNode with spec verification"
└─ npm test

EXPECTED BEHAVIOR:
Input: HyperAgentState
├─ Check all 7 fields present:
│ ├─ userInput ✓
│ ├─ policies ✓
│ ├─ generatedCode ✓
│ ├─ auditResults ✓
│ ├─ validationStatus ✓
│ ├─ deploymentHash ✓
│ └─ monitoringMetrics ✓
├─ Check types:
│ ├─ userInput: string ✓
│ ├─ policies: PolicyObject[] ✓

```
│  └─ etc.
├─ Check cross-field rules:
│  └─ deploymentHash not empty → is valid 0x... format ✓
└─ Output: validationStatus = true
```

Day 5: DeployNode

Friday, January 30

DeployNode: Blockchain deployment

```
├─ Create src/services/blockchain/provider.ts
│  ├─ Initialize Mantle RPC (primary)
│  ├─ Initialize Ethereum RPC (fallback)
│  ├─ Initialize Polygon RPC (fallback)
│  └─ Export provider selection function

├─ Create src/services/blockchain/mantle.ts
│  ├─ Mantle-specific deployment logic
│  ├─ Gas price estimation
│  ├─ Contract deployment
│  └─ Confirmation waiting

├─ Create src/services/blockchain/signer.ts
│  ├─ Sign transaction
│  ├─ Handle private key management
│  ├─ Verify signature

├─ Create src/nodes/deploy.ts
├─ Implement compileContract() function
│  ├─ Use solc compiler
│  ├─ Generate bytecode
│  └─ Extract ABI
├─ Implement signTransaction() function
│  ├─ Create ethers.js contract factory
│  ├─ Sign deployment transaction
│  └─ Return signed tx
├─ Implement submitToChain() function
│  ├─ Send to Mantle RPC
│  ├─ Track tx hash
│  ├─ Return transaction hash
│  └─ Handle network errors
├─ Implement waitForConfirmation() function
│  ├─ Poll blockchain for tx confirmation
│  ├─ Wait 12 blocks
│  ├─ Return contract address
│  └─ Handle timeout

├─ Create tests/unit/deploy.test.ts
│  ├─ Test compilation
│  ├─ Test signing (with test wallet)
```

```
│ ├─ Test submission (mock RPC)
│ └─ Test confirmation polling
```

Integration

```
├─ Update state-machine.ts
│ └─ Add DeployNode after ValidateNode
├─ Test end-to-end on testnet
├─ Commit: "Add DeployNode with blockchain integration"
└─ npm test
```

WEEK 2 COMPLETE CHECKPOINT:
✓ PolicyNode → working
✓ GenerateNode with LLM router → working
✓ AuditNode with Slither → working
✓ ValidateNode with type checking → working
✓ DeployNode with blockchain → working
✓ 90%+ test coverage
✓ Ready for Week 3 (MonitorNode + EigenDANode)

Week 3: Monitoring & Production (February 2-8)

Goal: Implement MonitorNode, EigenDANode, and deploy infrastructure

Day 1: MonitorNode

Monday, February 2

MonitorNode: Runtime tracking

```
├─ Create src/services/monitoring/prometheus.ts
│ ├─ Initialize Prometheus client
│ ├─ Define metrics:
│ │ ├─ contracts_generated (counter)
│ │ ├─ contracts_deployed (counter)
│ │ ├─ llm_calls_total (counter)
│ │ ├─ llm_cost_total (gauge)
│ │ ├─ deployment_gas_used (gauge)
│ │ ├─ audit_failures (counter)
│ │ └─ etc.
│ ├─ recordMetric() function
│ └─ getMetrics() function

├─ Create src/services/monitoring/grafana.ts
│ ├─ Connect to Grafana API
│ ├─ createDashboard() function
│ │ └─ Create dashboard with key charts
│ ├─ addChart() function
│ │ └─ Add time-series chart
```

```
│  └─ setAlerts() function
│  └─ Setup alerts (gas spike, failures, etc.)

├─ Create src/nodes/monitor.ts
├─ Implement startTracking() function
│  └─ Initialize background monitoring job
├─ Implement pollBlockchain() function
│  ├─ Query contract state every 30s
│  ├─ Track events emitted
│  ├─ Track gas usage
│  └─ Log errors
├─ Implement emitMetrics() function
│  ├─ Send metrics to Prometheus
│  ├─ Update Grafana dashboard
│  └─ Trigger alerts if needed
├─ Implement handleErrors() function
│  └─ Log & report on failures

├─ Create tests/unit/monitor.test.ts
│  ├─ Test metric recording
│  ├─ Test blockchain polling
│  ├─ Test error handling
│  └─ Test async behavior
```

Integration

```
├─ Update state-machine.ts
│  └─ Add MonitorNode after DeployNode
├─ Start background monitoring
├─ Commit: "Add MonitorNode with Prometheus/Grafana"
└─ npm test
```

Day 2: EigenDANode

Tuesday, February 3

EigenDANode: Immutable storage & proof

```
├─ Create src/nodes/eigenda.ts
├─ Implement serializeState() function
│  ├─ Convert HyperAgentState to JSON
│  ├─ Add timestamp & signature
│  └─ Return serialized string

├─ Implement uploadToIPFS() function
│  ├─ Use Pinata SDK
│  ├─ Upload serialized state
│  ├─ Get IPFS hash
│  └─ Return IPFS path (QmXxx...)
```

```
├─ Implement storeOnBlockchain() function
│  ├─ Create registry contract call
│  ├─ Store IPFS hash on Mantle
│  ├─ Get transaction hash
│  └─ Wait for confirmation

├─ Implement generateProof() function
│  ├─ Create merkle proof
│  ├─ Link IPFS hash to blockchain proof
│  └─ Return complete proof object

├─ Create tests/unit/eigenda.test.ts
│  ├─ Test serialization
│  ├─ Test IPFS upload (mock)
│  ├─ Test blockchain storage (mock)
│  └─ Test proof generation
```

Integration

```
├─ Update state-machine.ts
│  └─ Add EigenDANode as final step
├─ Test: Full end-to-end flow
│  └─ Input → PolicyNode → ... → EigenDANode → Output
├─ Commit: "Add EigenDANode with IPFS & blockchain storage"
└─ npm test

WEEK 3 PROGRESS:
✓ All 7 nodes implemented
✓ Monitoring operational
✓ Audit trail stored on IPFS + blockchain
✓ 90%+ test coverage
✓ Ready for final week (API + deployment)
```

Day 3-5: API Layer & Deployment

Wednesday-Friday, February 4-6

Day 3: REST API

```
├─ Create src/api/main.ts
│  ├─ Initialize Express.js server
│  ├─ Setup middleware
│  ├─ Setup error handling
│  └─ Start listening on port 3000

├─ Create src/api/routes.ts
│  ├─ POST /api/contracts/generate
│  │  └─ Body: { userInput, walletAddress, chain }
│  │  Returns: { contractAddress, txHash, auditTrail, ... }
│  ├─ GET /api/contracts/:id
│  │  └─ Returns: Full contract details + state
```

```
│  ├─ GET /api/contracts/:id/audit
│  │  └─ Returns: Complete audit report
│  ├─ GET /api/monitoring/:id
│  │  └─ Returns: Live monitoring metrics
│  └─ GET /api/health
│     └─ Returns: { status: "ok" }

├─ Create src/api/middleware.ts
│  ├─ validateSchema() → Zod validation
│  ├─ rateLimit() → 100 req/min per IP
│  ├─ authenticate() → Check API key
│  ├─ errorHandler() → Catch all errors
│  └─ requestLogger() → Log all requests

├─ Create tests/integration/e2e-flow.test.ts
│  └─ Test full API workflow
```

Day 4: Docker & Deployment Configuration

```
├─ Create Dockerfile
│  ├─ Base: node:20-alpine
│  ├─ Install dependencies
│  ├─ Build TypeScript
│  ├─ Expose port 3000
│  └─ Run start command

├─ Create docker-compose.yml
│  ├─ hyperagent service
│  ├─ postgres service (optional)
│  ├─ redis service (optional)
│  ├─ prometheus service
│  ├─ grafana service
│  └─ chroma service

├─ Create render.yaml
│  ├─ Define Render deployment
│  ├─ Environment variables
│  ├─ Scaling configuration
│  └─ Health check endpoints
```

Day 5: Documentation & Final Testing

```
├─ Update all 7 documentation files
│  ├─ Reflect implementation
│  ├─ Update examples
│  ├─ Update architecture diagrams
│  └─ Create runbooks

├─ Run final test suite
│  ├─ npm test (should be 90%+ coverage)
```

```
│ ├─ npm run lint (no errors)
│ ├─ npm run build (no errors)
│ └─ Smoke test API endpoints

├─ Deploy to staging
│ ├─ docker-compose up
│ ├─ curl http://localhost:3000/api/health
│ └─ Run full E2E test

├─ Final commit
│ └─ "Complete implementation: 7 nodes + API + monitoring"
```

Type System & State Management

HyperAgentState Definition

// src/types/state.ts

import { PolicyObject, GeneratedCode, AuditReport, MonitoringMetrics } from '.';

/**

- HyperAgentState: Core immutable state object
-
- This state object MUST have exactly 7 fields:
-
  1. userInput - User's natural language request
-
  2. policies - Extracted requirements from input
-
  3. generatedCode - Generated smart contract code
-
  4. auditResults - Security audit report
-
  5. validationStatus - Whether state is valid (all 7 fields present + correct types)
-
  6. deploymentHash - Transaction hash from blockchain deployment
-
  7. monitoringMetrics - Runtime metrics (gas, events, errors)
-
- CRITICAL: All 7 fields must ALWAYS be present.
- No field can be undefined or null.
- No extra fields can be added.
-
- This is "specification-locked" - the TypeScript compiler enforces it.
  */
  export interface HyperAgentState {
  // Field 1: User Input
  userInput: string;
  // Example: "Create ERC-20 with 10M max supply and 25% burn"

```
// Field 2: Extracted Policies
policies: PolicyObject[];
// Example: [
// { type: "erc-20", version: "latest" },
// { type: "max-supply", value: 10000000 },
// { type: "burn-rate", value: 0.25 }
// ]

// Field 3: Generated Code
generatedCode: string;
// Example: "pragma solidity ^0.8.0; contract Token { ... }"

// Field 4: Audit Results
auditResults: AuditReport;
// Example: {
// status: "passed",
// severity: "none",
// issues: [],
// timestamp: 1705600000
// }

// Field 5: Validation Status
validationStatus: boolean;
// Example: true (all 7 fields present + types correct)

// Field 6: Deployment Hash
deploymentHash: string;
// Example: "0xabcd1234..." (transaction hash from blockchain)

// Field 7: Monitoring Metrics
monitoringMetrics: MonitoringMetrics;
// Example: {
// gasUsed: 2500000,
// events: [],
// errors: [],
// timestamp: 1705600000
// }
}

/**

- Verification function: Ensure state has exactly 7 fields
- Runs at compile-time via TypeScript strict mode
- Runs at runtime via this function
  */
  export function verifyStateShape(state: unknown): state is HyperAgentState {
  if (!state || typeof state !== 'object') {
  throw new Error('State must be an object');
  }

const obj = state as Record<string, unknown>;
```

```
// Check: exactly 7 keys
const keys = Object.keys(obj);
if (keys.length !== 7) {
throw new Error(State must have exactly 7 fields, got ${keys.length});
}

// Check: all required keys present
const required = ['userInput', 'policies', 'generatedCode', 'auditResults', 'validationStatus',
'deploymentHash', 'monitoringMetrics'];
for (const key of required) {
if (!(key in obj)) {
throw new Error(Missing required field: ${key});
}
}

// Check: no extra keys
for (const key of keys) {
if (!required.includes(key)) {
throw new Error(Unexpected field: ${key}. State can only have: ${required.join(', ')});
}
}

// Check: no null/undefined fields
for (const key of required) {
if (obj[key] === null || obj[key] === undefined) {
throw new Error(Field "${key}" cannot be null or undefined);
}
}

return true;
}

/**

- Factory function: Create new HyperAgentState
  */
  export function createHyperAgentState(
  userInput: string
  ): HyperAgentState {
  return {
  userInput,
  policies: [],
  generatedCode: '',
  auditResults: {
  status: 'pending',
  severity: 'none',
  issues: [],
  timestamp: Date.now()
  },
  validationStatus: false,
  deploymentHash: '',
  monitoringMetrics: {
```

```
gasUsed: 0,
events: [],
errors: [],
timestamp: Date.now()
}
};
}
```

/**

- Immutable state update: Return new state with field updated

```
*/
export function updateState<K extends keyof HyperAgentState>(
state: HyperAgentState,
field: K,
value: HyperAgentState[K]
): HyperAgentState {
// Verify new state would still have exactly 7 fields
const newState = { ...state, [field]: value };
verifyStateShape(newState);
return newState;
}
```

Node Implementation Templates

Template 1: Simple Node (PolicyNode)

```
// src/nodes/policy.ts

import { HyperAgentState, updateState } from '../types/state';
```

/**

- PolicyNode: Parse user input into structured policies
-
- Input: state.userInput (string)
- Output: state.policies (array of policy objects)

```
*/
export async function policyNode(
state: HyperAgentState
): Promise<HyperAgentState> {
console.log([PolicyNode] Processing: "${state.userInput}");

// Step 1: Parse user input with regex
const policies = parseUserInput(state.userInput);

// Step 2: Update state with policies
return updateState(state, 'policies', policies);
}
```

/**

- Extract policies from natural language user input
  */
  function parseUserInput(input: string): PolicyObject[] {
  const policies: PolicyObject[] = [];

```
// Pattern 1: ERC standard detection
if (/erc-?20/i.test(input)) {
policies.push({ type: 'erc-20', version: 'latest' });
}
if (/erc-?721/i.test(input)) {
policies.push({ type: 'erc-721', version: 'latest' });
}

// Pattern 2: Supply limits
const supplyMatch = input.match(/(\d+[KMB]?)\s*(?:max )?supply/i);
if (supplyMatch) {
const value = parseNumber(supplyMatch[1]);
policies.push({ type: 'max-supply', value });
}

// Pattern 3: Burn rate
const burnMatch = input.match(/(\d+)%\s*burn/i);
if (burnMatch) {
const value = parseInt(burnMatch[1]) / 100;
policies.push({ type: 'burn-rate', value });
}

// Pattern 4: Minting controls
if (/pause|pausable|pauseable/i.test(input)) {
policies.push({ type: 'pausable', value: true });
}
if (/mint|minting/i.test(input)) {
policies.push({ type: 'mintable', value: true });
}

return policies;
}

/**
```

- Parse numeric values with unit suffixes (1M = 1000000)
  */
  function parseNumber(str: string): number {
  const multipliers: Record<string, number> = {
  'K': 1_000,
  'M': 1_000_000,
  'B': 1_000_000_000
  };

```
const match = str.match(/(\d+)([KMB])?/i);
if (!match) return 0;
```

```
const num = parseInt(match[1]);
const unit = (match[2] || '').toUpperCase();
return num * (multipliers[unit] || 1);
}
```

Template 2: LLM Node (GenerateNode)

```
// src/nodes/generate.ts

import { HyperAgentState, updateState } from '../types/state';
import { routeLLMCall } from '../services/llm/router';
import { queryChroma, cacheInChroma } from '../memory/vector-store';

/**

- GenerateNode: Generate smart contract code
-
- Input: state.policies (array of policies)
- Output: state.generatedCode (Solidity code)
-
- Process:
-
    1. Query Chroma for similar contracts
-
    2. Call LLM with fallback chain
-
    3. Validate output is valid Solidity
-
    4. Cache result
       */
       export async function generateNode(
       state: HyperAgentState
       ): Promise<HyperAgentState> {
       console.log([GenerateNode] Generating code from policies:, state.policies);

try {
// Step 1: Query similar contracts from Chroma
const similarContracts = await queryChroma(state.policies);
console.log([GenerateNode] Found ${similarContracts.length} similar contracts);

// Step 2: Build LLM prompt
const prompt = buildPrompt(state.policies, similarContracts);

// Step 3: Call LLM with fallback chain
const generatedCode = await routeLLMCall(prompt);
console.log(`[GenerateNode] Generated code (${generatedCode.length} chars)`);

// Step 4: Validate Solidity syntax
```

```
validateSolidity(generatedCode);
console.log(`[GenerateNode] Code validation passed`);

// Step 5: Cache in Chroma for future reference
await cacheInChroma(generatedCode, state.policies);

// Step 6: Update state
return updateState(state, 'generatedCode', generatedCode);
```

} catch (error) {
console.error([GenerateNode] Error:, error);
throw new Error(GenerateNode failed: ${error});
}
}

/**

- Build LLM prompt with policies and similar contracts
  */
  function buildPrompt(
  policies: PolicyObject[],
  similarContracts: ContractExample[]
  ): string {
  const policyText = policies
  .map(p => - ${p.type}: ${JSON.stringify(p.value || true)})
  .join('\n');

const examplesText = similarContracts
.map(c => \``solidity\n${c.code}\n```)
.join('\n\n');

return `
You are an expert Solidity smart contract developer.

Generate a production-ready Solidity smart contract based on these requirements:

${policyText}

Here are similar examples for reference:

${examplesText}

Generate only the contract code. No explanations. Start with pragma and end with closing brace.
`;
}

/**

- Validate that output is valid Solidity
  */
  function validateSolidity(code: string): void {
  // Check: starts with pragma
  if (!code.trim().startsWith('pragma')) {
  throw new Error('Code must start with pragma');
  }

// Check: contains contract keyword
if (!/\bcontract\s+\w+/i.test(code)) {
throw new Error('Code must define a contract');
}

// Check: balanced braces
const open = (code.match(/{/g) || []).length;
const close = (code.match(/}/g) || []).length;
if (open !== close) {
throw new Error(Unbalanced braces: ${open} open, ${close} close);
}
}

Testing Strategy

Unit Test Template

// tests/unit/policy.test.ts

import { policyNode, createHyperAgentState, verifyStateShape } from '../../src';

describe('PolicyNode', () => {

describe('parseUserInput', () => {

```
it('should extract ERC-20 policy', async () => {
  const state = createHyperAgentState('Create ERC-20 token');
  const result = await policyNode(state);

  expect(result.policies).toContainEqual(
    expect.objectContaining({ type: 'erc-20' })
  );
});

it('should extract max supply policy', async () => {
  const state = createHyperAgentState('Create token with 10M max supply');
  const result = await policyNode(state);
```

```
    expect(result.policies).toContainEqual(
      expect.objectContaining({ type: 'max-supply', value: 10000000 })
    );
  });

  it('should handle multiple policies', async () => {
    const state = createHyperAgentState(
      'Create ERC-20 with 10M max supply and 25% burn'
    );
    const result = await policyNode(state);

    expect(result.policies.length).toBeGreaterThanOrEqual(3);
  });
```

});

describe('state verification', () => {

```
  it('should maintain exactly 7 state fields', async () => {
    const state = createHyperAgentState('Test input');
    const result = await policyNode(state);

    expect(Object.keys(result).length).toBe(7);
    verifyStateShape(result); // Should not throw
  });

  it('should not add extra fields', async () => {
    const state = createHyperAgentState('Test input');
    const result = await policyNode(state);

    const allowedFields = [
      'userInput',
      'policies',
      'generatedCode',
      'auditResults',
      'validationStatus',
      'deploymentHash',
      'monitoringMetrics'
```

```
    ];

    Object.keys(result).forEach(key => {
      expect(allowedFields).toContain(key);
    });
  });

});
});
```

Integration Test Template

```
// tests/integration/e2e-flow.test.ts

import { createWorkflow } from '../../src/protocols/state-machine';

describe('End-to-End Flow', () => {

it('should complete full workflow from input to deployment', async () => {
const workflow = await createWorkflow();

  const result = await workflow.run({
    userInput: 'Create ERC-20 with minting limits',
    walletAddress: '0x123...',
    chain: 'mantle-testnet'
  });

  // Verify all 7 state fields populated
  expect(result).toHaveProperty('userInput');
  expect(result).toHaveProperty('policies');
  expect(result).toHaveProperty('generatedCode');
  expect(result).toHaveProperty('auditResults');
  expect(result).toHaveProperty('validationStatus');
  expect(result).toHaveProperty('deploymentHash');
  expect(result).toHaveProperty('monitoringMetrics');

  // Verify values make sense
  expect(result.userInput).toBeTruthy();
  expect(result.policies.length).toBeGreaterThan(0);
  expect(result.generatedCode).toMatch(/pragma solidity/);
```

```
  expect(result.deploymentHash).toMatch(/^0x[0-9a-fA-F]+$/);
  expect(result.validationStatus).toBe(true);
```

}, 60000); // 60 second timeout
});

Deployment Architecture

Production Deployment Diagram

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ RENDER DEPLOYMENT │
│ (Auto-scaling PaaS platform) │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
┌──────────────────┴──────────────────┐
│ │
▼ ▼
┌──────────────────────────┐ ┌──────────────────────────┐
│ HyperAgent │ │ HyperAgent │
│ Service #1 │ │ Service #2 │
│ Node.js 20 │ │ Node.js 20 │
│ Port 3000 │ │ Port 3000 │
└──────────────────────────┘ └──────────────────────────┘
│ │
├──────────────────────────────────────┤
│ │
▼ ▼
┌──────────────────────────────────────────────────────────┐
│ Load Balancer / Health Check │
│ (Render routing layer) │
└──────────────────────────────────────────────────────────┘
│
┌──────────────────────┴──────────────────────┐
│ │
▼ ▼
┌──────────────────────────┐ ┌──────────────────────────┐
│ PostgreSQL DB │ │ Redis Cache │
│ (Render PaaS) │ │ (Render PaaS) │
│ Backups: daily │ │ TTL: 24h │
└──────────────────────────┘ └──────────────────────────┘
│ │
├──────────────────────────────────────────────┤
│ │
▼ ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│ External Services (Async) │
```

```
│ │
│ ├─ Chroma (Vector DB) │
│ ├─ Pinata IPFS (Storage) │
│ ├─ LLM APIs (Claude, Gemini, GPT-4) │
│ ├─ Blockchain RPCs (Mantle, Ethereum, Polygon)│
│ └─ Slither (Security Analysis) │
└──────────────────────────────────────────────────────────┘
│
▼
┌──────────────────────────────────────────────────────────┐
│ Monitoring & Observability │
│ │
│ ├─ Prometheus (metrics collection) │
│ ├─ Grafana (dashboard visualization) │
│ ├─ Sentry (error tracking) │
│ └─ CloudFlare (DDoS protection) │
└──────────────────────────────────────────────────────────┘
```

Cost & Resource Tracking

Cost Model

```
// src/utils/cost-calculator.ts

export interface CostBreakdown {
llm: number; // LLM API calls
blockchain: number; // Gas fees
storage: number; // IPFS/database
monitoring: number; // Prometheus/Grafana
infrastructure: number; // Hosting
total: number;
}

export function calculateCost(
state: HyperAgentState,
config: CostConfig
): CostBreakdown {

// LLM Cost
const llmCost = calculateLLMCost(state);
// Typical: $0.004 - $0.020 depending on LLM fallbacks

// Blockchain Cost (gas)
const blockchainCost = calculateGasCost(state);
// Typical: $0.10 - $5.00 depending on chain and contract complexity

// Storage Cost
const storageCost = calculateStorageCost(state);
// Typical: $0.001 - $0.010

// Monitoring (fixed)
const monitoringCost = 0.001; // Negligible
```

```
// Infrastructure (fixed per request)
const infrastructureCost = calculateInfrastructureCost();
// Typical: $0.001 - $0.005

const total = llmCost + blockchainCost + storageCost + monitoringCost + infrastructureCost;

return {
llm: llmCost,
blockchain: blockchainCost,
storage: storageCost,
monitoring: monitoringCost,
infrastructure: infrastructureCost,
total
};
}
```

Cost Breakdown (per contract):
LLM Cost: $0.004 - $0.020 (fallback chain)
Blockchain Cost: $0.100 - $5.000 (depends on chain)
Storage Cost: $0.001 - $0.010 (IPFS + metadata)
Infrastructure: $0.001 - $0.005 (hosting + compute)
———————————————————————————————
TOTAL: $0.106 - $5.035

Monthly Costs (at scale):
1,000 contracts/month:
├─ LLM: $4 - $20
├─ Blockchain: $100 - $5,000
├─ Storage: $1 - $10
├─ Monitoring: $0
├─ Infrastructure: $200/month (Render + DB)
└─ TOTAL: ~$306 - $5,230/month

Pricing model: $10/contract minimum
Gross margin: 99%+

Summary & Next Steps

Completion Criteria (All 15 Items Must Be Checked)

- [ ] HyperAgentState has exactly 7 fields
- [ ] All 7 fields always present (no undefined/null)
- [ ] No extra fields (spec-locked by TypeScript)
- [ ] PolicyNode implemented & tested
- [ ] GenerateNode with LLM fallback chain
- [ ] AuditNode with Slither integration
- [ ] ValidateNode with type verification
- [ ] DeployNode with blockchain integration
- [ ] MonitorNode with Prometheus metrics
- [ ] EigenDANode with IPFS + blockchain storage
- [ ] REST API fully functional

- [ ] 90%+ test coverage
- [ ] Documentation updated (all 7 files)
- [ ] Production deployment working
- [ ] Cost tracking implemented

Success Metrics

- **Timeline**: February 15, 2026 (4 weeks)
- **Code Quality**: 90%+ test coverage, TypeScript strict mode
- **Performance**: <2 second response time for contract generation
- **Reliability**: 99%+ uptime on monitoring
- **Cost**: <$5 per contract (including gas)
- **Auditability**: 100% (immutable trail on IPFS + blockchain)

References

Documentation Files:

1. `hyperagent_dna_blueprint.md` - Node specifications
2. `hyperagent_playbook.md` - Day-by-day timeline
3. `hyperagent_system_prompts.md` - LLM safety
4. `hyperagent_tech_stack_architecture.md` - Tech choices
5. `tech_stack_blueprint_integration.md` - Technology mapping
6. `documentation_index.md` - Navigation guide
7. `blueprint_executive_summary.md` - Quick reference
8. `complete_system_guide.md` - Full guide

Key Technologies:

- TypeScript 5.x + Node.js 20 LTS
- LangGraph for state machine
- Claude Opus/Gemini/GPT-4 for code generation
- Slither for security auditing
- ethers.js for blockchain integration
- Chroma for vector embeddings
- Pinata for IPFS storage
- Prometheus + Grafana for monitoring
- Render PaaS for deployment

Document Version: 1.0
Last Updated: January 18, 2026
Status: Ready for Execution
Next Milestone: January 19, 2026, 9:00 AM (Week 1 kickoff)